Made by Jair with google gemini pro on 06/04/2026, for more information:
jairmarttila725 at gmail dot com

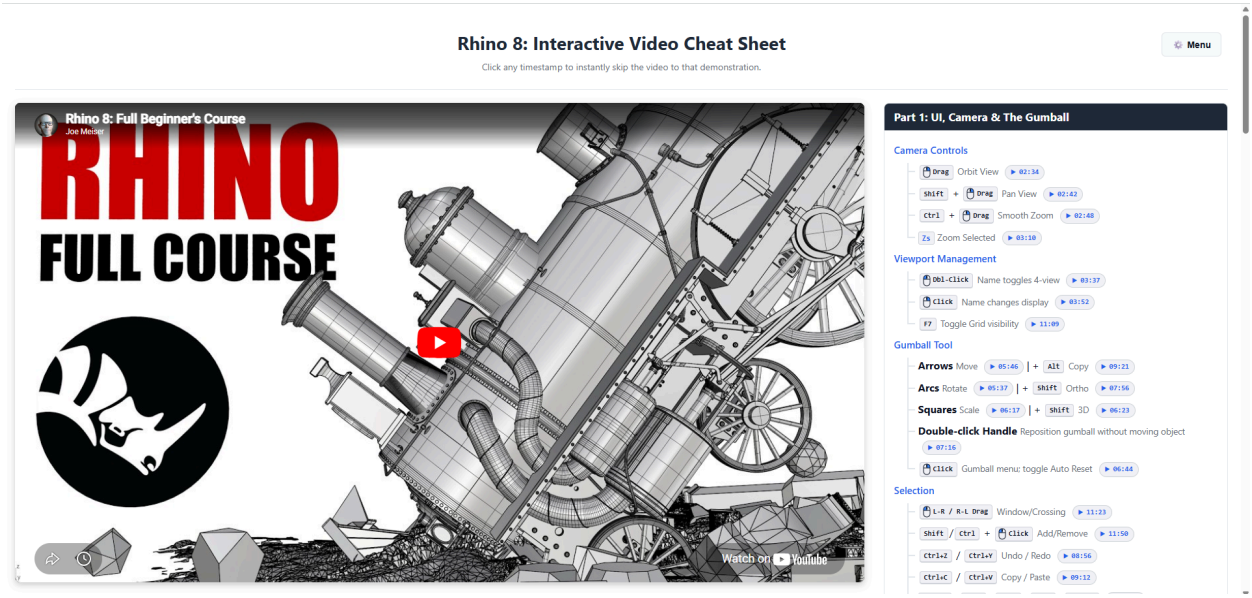
Youtube video referred: Rhino 8: Full Beginner's Course

Download practice files from: https://joemeister0.gumroad.com/l/azptap

Rhino 8 Interactive Cheat Sheet: Setup, Code Guide, and Deployment

This document explains how to launch the single-file web application and how to tweak the code to fit your exact needs.

1. Overview of Functions



At its core, this HTML file is a **Single-Page Application (SPA)**. Instead of just displaying static text, it reacts to your inputs and controls a live video. Here are its main functions:

1. **Interactive Video Control:** It uses the YouTube IFrame Player API. When you click a timestamp, the JavaScript calculates the exact second and forces the embedded video to jump to that moment and play automatically.
2. **Responsive "Sticky" Layout:** Using CSS Flexbox and Grid, the layout adapts to your screen. On mobile, the video sits at the top and scrolls away. On a desktop monitor, the video "sticks" to the left side of your screen so you can watch it while scrolling through the massive list of commands on the right.
3. **Dynamic Theming & Sizing:** A built-in settings menu allows you to flip a switch to inject a "Dark Mode" CSS class into the page, changing all the colors instantly. A slider dynamically alters a CSS variable to make the video panel wider or narrower.
4. **Print Optimization:** It includes a @media print CSS block. If you try to print the page, the code intentionally hides the video, the dark background, and the buttons, stripping it down

to pure text to save printer ink.

---

2. Getting Started: How to Run the File

Because this file uses the YouTube IFrame API to control a video via JavaScript, it cannot be run by simply double-clicking the .html file on your computer (this results in the browser address bar showing file:///... and triggers YouTube's **Error 153** security block).

You must serve the file through a local web server. Here are the easiest ways to do that:

Method A: Visual Studio Code (Recommended)

If you use VS Code for editing HTML:

5. Open VS Code and navigate to the **Extensions** tab on the left sidebar.
6. Search for and install **Live Server** (by Ritwick Dey).
7. Open your .html file in VS Code.
8. Right-click anywhere in the code editor and select **Open with Live Server**.
9. Your default browser will open automatically, and the video will work perfectly.

Method B: CodePen (No Installation)

If you don't want to deal with local files at all:

1. Go to CodePen.io/pen.
2. Paste the entire HTML code into the **HTML box**.
3. The app will run instantly in the preview window below.

Method C: Python (Mac/Windows)

If you have Python installed on your machine:

1. Open your Terminal (Mac) or Command Prompt (Windows).
2. Navigate to the folder where your HTML file is saved using the cd command (e.g., cd Desktop/RhinoFolder).
3. Type python -m http.server (or python3 -m http.server on Mac) and press **Enter**.
4. Open your web browser and navigate to http://localhost:8000. Click on your HTML file.

   Super easy when you have python setup correctly!

Method D: ngrok (Public Access to Local Server)

If you have a local server running (for example, on port 8000 using Python) but want to give someone else temporary public access without hosting the file permanently, you can use ngrok to create a secure tunnel.

5. Start your local server as described in Method C:

```shell
python -m http.server 8000
```

6. Open a second terminal window and run the following command to expose your local port 8000 to the internet:

```shell
ngrok http 8000
```

7. ngrok will provide a public URL (e.g., https://random-id.ngrok-free.app). Anyone with this link can now view your app and the video will function correctly.

Method E: Permanent web hosting

If you want a permanent, public link that you can share with others, you can host the file on a web server. This is the most professional method.

1. **Use a hosting service:** Upload the rhino_hotkeys_with_timestamps_interactive.html file to a web hosting service (like Vercel, Netlify, or a traditional web host via FTP).
2. **Ensure it's public:** The file should be placed in the public-facing directory (often named public_html or www).
3. **Access the URL:** Navigate to the resulting URL (e.g., https://yourdomain.com/rhino_hotkeys_with_timestamps_interactive.html). The YouTube video will function correctly because it is being served over HTTP/HTTPS.

---

3. Step-by-Step Code Explanation

The code is divided into three standard web development sections: <style> (Design), <body> (Structure), and <script> (Logic).

1. The CSS Styling (<style>)

This section handles the entire visual look of the app.

8. **CSS Variables (:root):** At the very top, colors are stored in variables like --bg-color and --accent. This makes it incredibly easy to change the color scheme later.
9. **Dark Theme (body.dark-theme):** When dark mode is activated, this block simply overwrites the variables in :root. For example, --bg-color changes from white to dark gray.
10. **The Sticky Video (.video-panel):** This uses position: sticky; top: 0;. This tells the browser:

"When the user scrolls down, lock this element to the top of the screen so it never disappears."

11. **UI Elements (kbd, .mouse-action, .ts):** These classes use borders, padding, and subtle shadows to make plain text look like physical keyboard keys, mouse buttons, and clickable timestamp pills.
12. **Desktop Layout (@media (min-width: 960px)):** This is a "media query." It tells the browser that if the screen is wider than 960 pixels, it should switch the .layout-container from stacking vertically to aligning side-by-side.

2. The HTML Structure (<body>)

This is the skeleton of the page.

4. **Hidden SVG Icons (<svg><defs>):** At the top of the body, there is an invisible SVG block. By defining the "mouse" shape once in the <defs> tag, the code can reuse it dozens of times throughout the document using <use href="#mouse-base"/>. This keeps the file size small and loads instantly.
5. **The Layout Split:** The main content is wrapped in . Inside this, there are two main chunks:
    - : Holds the empty where YouTube will eventually inject the video.
    - : Holds all the cheat sheet text, structured as nested bullet points (<ul> and <li>).
6. **The Timestamps:** Every timestamp looks like this: <a class="ts" data-time="02:34">02:34</a>. The visible text is for the human; the hidden data-time attribute is what the JavaScript will read later.
7. **The Modal (#menuModal):** At the bottom of the HTML is the popup window code. It defaults to display: none; (invisible). It contains the tabs for Info, Settings, and the Chat interface.

3. The JavaScript Logic (<script>)

This is the brain of the operation, making everything interactive.

- **YouTube Initialization:** * onYouTubeIframeAPIReady(): This function is triggered automatically by the external YouTube script. It finds the and replaces it with the Rhino 8 video using the videoId.
- **Timestamp Logic:**
    - parseTimeStr(timeStr): Computers don't understand "1:30" as time. This custom function splits the text at the colon, multiplies the minutes by 60, and returns pure seconds (e.g., 90 seconds).
    - **The Click Event:** The script grabs every element with the class .ts. When you click one, it prevents the page from reloading (e.preventDefault()), reads the data-time attribute, converts it to seconds, and tells the YouTube player to seekTo that second.
- **Modal & Tab Switching:**

    - Simple event listeners check when you click the "Menu" button and change the modal's CSS from display: none to display: flex (making it appear).
    - When you click a tab, the script loops through all tab content, hides them all, and then only reveals the one you clicked.
- **Settings Controls:**
    - **Dark Mode:** When you click the toggle, JS simply adds or removes the .dark-theme class from the <body> tag. The CSS handles the rest.
    - **Video Size:** When you move the slider, JS grabs the value (e.g., 60) and updates the CSS variable --video-width to 60%.
- **Mock Chat Logic:**
    - When you hit "Send", handleSend() grabs your text, creates a new element containing your message, and attaches it to the chat window.
    - It then uses setTimeout to wait exactly 1 second (1000 milliseconds) before injecting a pre-written automated response from the "bot."

---

4. Examples of making Adjustments to the Code

The code is contained entirely in one file, split into three main sections: <style> (CSS/Design), <body> (HTML/Structure), and <script> (JavaScript/Logic). Here is how to modify it.

Changing the YouTube Video

If you want to use this template for a different tutorial video:

2. Scroll down to the <script> section near the very bottom of the file.
3. Look for the onYouTubeIframeAPIReady function.
4. Change the videoId string to the ID of your new YouTube video.
   (Note: The ID is the random string of letters and numbers in a YouTube URL after v=).

JavaScript
```javascript
function onYouTubeIframeAPIReady() {
    player = new YT.Player('player', {
        videoId: 'NEW_VIDEO_ID_HERE', // <-- Change this
        playerVars: { 'playsinline': 1, 'rel': 0 }
    });
```

```
}
```

Adding a New Command and Timestamp

To add a new item to the list, scroll to the HTML <ul> (unordered list) sections. You can copy and paste an existing line <li> and change the text and time.

Template:

```html
<li>
    <span class="cmd-type">YourCommand</span>
    <span class="desc">What it does</span>
    <a class="ts" data-time="12:34">12:34</a>
</li>
```

- **Important:** The data-time attribute is what the JavaScript reads to skip the video. It accepts formats like MM:SS (e.g., 05:30) or H:MM:SS (e.g., 1:15:20).

Changing the Color Scheme

All the colors are controlled by CSS Variables at the very top of the <style> section. You can easily re-brand the whole page by changing these hex codes.

2. Scroll to the top of the <style> block and find :root.
3. To change the main accent color (the blue buttons/links), change --accent.
4. To change the dark header bar, change --primary.

```css
:root {
    --primary: #1f2937;    /* Dark gray header */
    --accent: #2563eb;     /* Blue timestamps and highlights */
    --bg-light: #f9fafb;   /* Light gray backgrounds */
}
```

Note: If you want to change the Dark Mode colors, look just below :root for the body.dark-theme block and change the variables there.

Adjusting the Default Video Size

By default, the video takes up 55% of the screen width on desktop. If you want it smaller or larger by default:

2. Go to the :root CSS variables and change --video-width: 70%; to your preferred percentage

(e.g., 40%).

3. To make sure the settings slider matches your new default, find the slider in the HTML (id="videoSizeSlider") and change its value attribute to match:
<input type="range" id="videoSizeSlider" min="40" max="80" value="40">

Modifying the Keyboard or Mouse Icons

If you want to add a new keyboard key, just wrap the text in <kbd> tags:

```html
<kbd>Ctrl</kbd> + <kbd>Shift</kbd> + <kbd>Z</kbd>
```

To insert a mouse icon, use this exact snippet, changing #mouse-lmb (Left Mouse Button) to #mouse-rmb (Right) or #mouse-mmb (Middle) depending on what you need:

```html
<span class="mouse-action">
    <svg class="mouse" viewBox="0 0 24 36"><use
href="#mouse-lmb"/><use href="#mouse-base"/></svg> Click
</span>
```

Adjusting the Menu (Popup) Window Size

To make the "⚙️ Menu" popup wider, taller, or smaller, look for the .modal class in the CSS (around line 105).

Find this block:

```css
.modal {
    background: var(--modal-bg); color: var(--text-color);
    width: 90%; max-width: 600px; max-height: 80vh;
border-radius: 12px;
    /* ... other settings ... */}
```

How to change it:

10. **To make it wider:** Increase the max-width. For example, change max-width: 600px; to max-width: 800px; or even 1000px.
11. **To make it taller:** Increase the max-height. Currently, it takes up 80% of the screen height (80vh). You can change this to 90vh or remove it entirely if you want it to stretch as far as the content needs.

Controlling Page Width and Margins

The overall width of the app is controlled in the body CSS. By adjusting the max-width, you determine how much horizontal space the cheat sheet occupies.

```CSS
body {
    /* Change to 100% for full-screen or a specific pixel value like 1200px for a centered column */
    max-width: 1600px;
    margin: 0 auto; /* Keeps the content centered if max-width is less than screen width */
    padding: 20px; }
```

Switching to a Multi-Column Layout

To maximize screen space on large monitors, you can force the cheat sheet sections to display in two parallel columns. This is controlled by the CSS Grid settings for the content container.

```CSS
.grid-2col
{ display: grid; grid-template-columns: 1fr; gap: 15px; }
/* Two-column layout for sections, change to 1fr 1fr if two-column layout is desired */
```

Modifying the Chat Bot's Automated Response

The bot's response is hardcoded (pre-written) within the JavaScript. To change what the bot says, find the handleSend() function within the <script> section and modify the string variable that holds the automated reply.

The automated response is often defined like this:

```JavaScript
function handleSend() {
    // ... user message logic

    // The pre-written automated response is often defined here
```

```
    const botResponse = "That’s a great question! For detailed
help on that command, please check the official Rhino
documentation.";

    // ... The code then inserts this response into the chat
after 1 second
    setTimeout(() => {
        // ...
        chatContainer.appendChild(createMessage('bot',
botResponse));
    }, 1000);
}
```

Change the text within the botResponse variable to customize the bot's reply.

Issues with manual? Contact jairmarttila725 at gmail dot com